시스템 프로그래밍

7장. 파일 및 레코드 잠금

컴퓨터소프트웨어학부

School of Computer Software at Daegu Catholic University © 2021

목차

- 실습 문제
  - ✓ 1. 레코드 영역 잠금
  - ✓ 2. 레코드 영역 동시 수정
  - ✓ 3. 권고 잠금과 강제 잠금
- 연습문제
  - ✓ 1. 원하는 잠금을 설정할 수 있는 함수 locks()를 작성
  - ✓ 2. 원하는 잠금을 검사할 수 있는 함수 lockt()를 작성

추가 세팅

- 4장 실습 문제 5번 레코드 저장을 통한 데이터 기록 필요
  - ✓이후 모든 작업은 레코드 “student”로 진행함.
  - ✓Student.h 는 4장의 Student.h을 기반으로 실습 진행

```
/home/snslab/CLionProjects/week4-5/cmake-build-debug/week4_5 student
학번    이름   점수
17124005 Jeon 80
21311659 Geon 90
11111111 Test 60
^D

Process finished with exit code 0
```

실습 문제 1 – 레코드 잠금

- 레코드 파일 읽기, 수정을 동시에 진행하여 레코드 영역 잠금 동작 결과에 대하여 확인하시오.
  - ✓ 참조 자료 : 강의자료 12번 슬라이드
  - ✓ 단, rdlock.c, wrlock.c 은 동일한 레코드 파일을 읽어야 한다.

1. rdlock.c, wrlock.c 파일을 작성하고 실행하시오.
2. wrlock 파일 실행을 통해 저장된 레코드 영역에 접근하세요.
3. 2번이 실행되고 있는 상황에서 rdlock을 실행하여 rwlock 에서 수정하고 있는 학번을 검색하여 그 결과를 확인하시오.
4. 예외처리
   - Argument 개수가 2개 이하일 때 프로그램 종료
   - 파일 잠금 처리를 할 수 없을 경우 경로 출력 후 종료

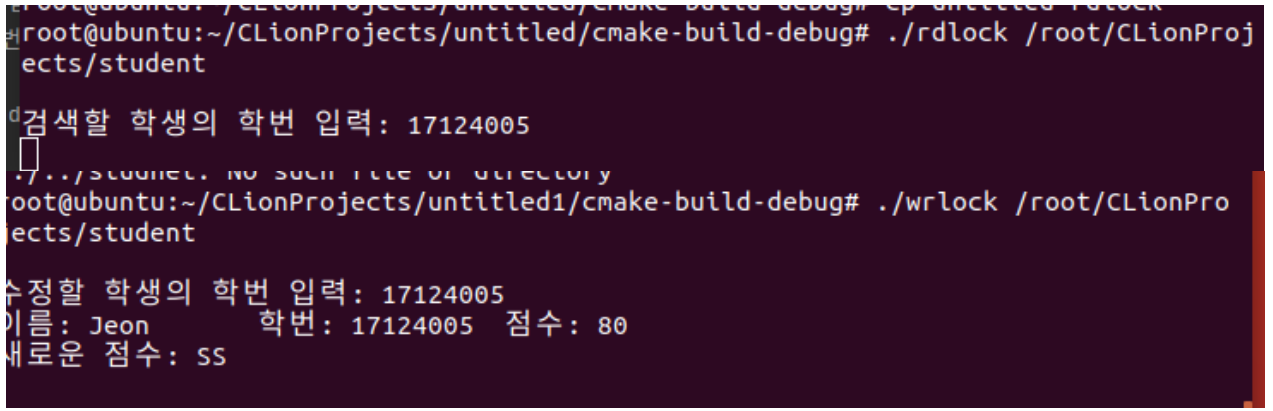
❖ 실행 명령어

- gcc –o rdlock rdlock.c
- gcc –o wrlock wrlock.c
- ./rdock student
- ./wrlock student

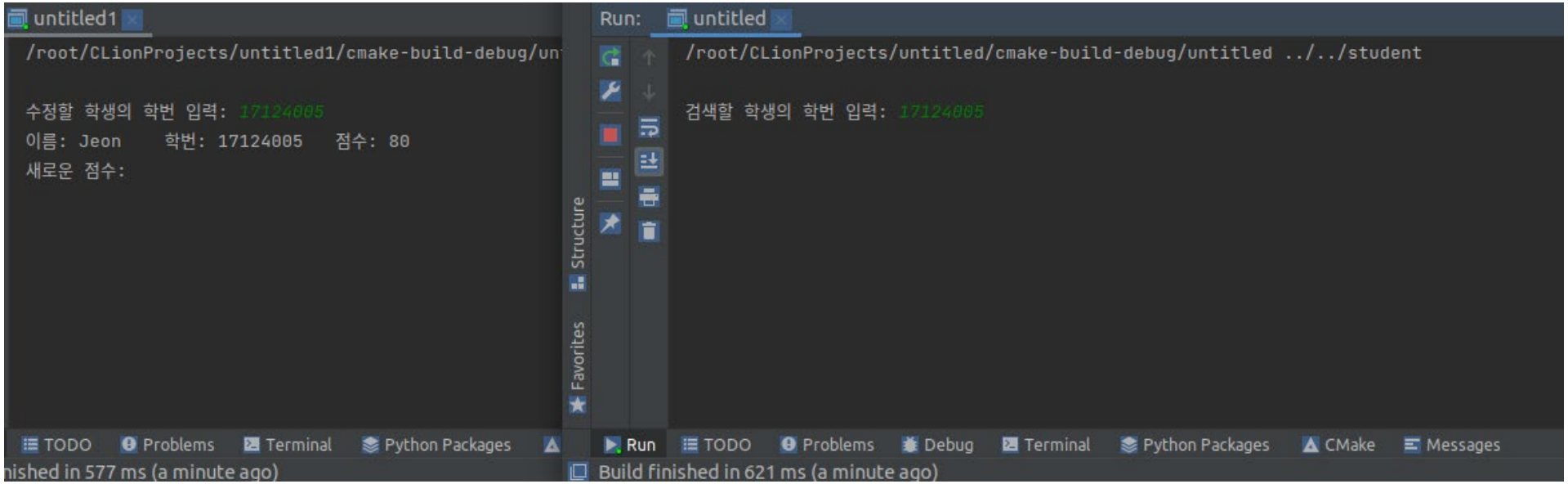
실습 문제 1 – 파일 열기

- 실행 결과

Terminal GCC



CLion



실습 문제 2 – 레코드 영역 동시 수정

- 레코드 파일 읽기, 수정을 동시에 진행하여 레코드 영역 잠금 동작 결과에 대하여 확인하시오.

✓참조 자료 : 강의자료 20번 슬라이드

1. wrlockf.c 파일을 작성하고 터미널을 사용하여 wrlockf 를 2개 실행하시오.
2. 실행한 첫번째의 wrlockf 프로그램에서 수정할 학생의 학번을 작성하세요.
3. 두번째 wrlockf 프로그램에서 첫번째 프로그램에서 수정하고 있는 학생의 학번에 접근하세요.
4. 예외처리
   - lockf() 함수를 사용하여 파일 잠금 처리를 수행. 만약, 잠금을 할 수 없다면 -1을 반환

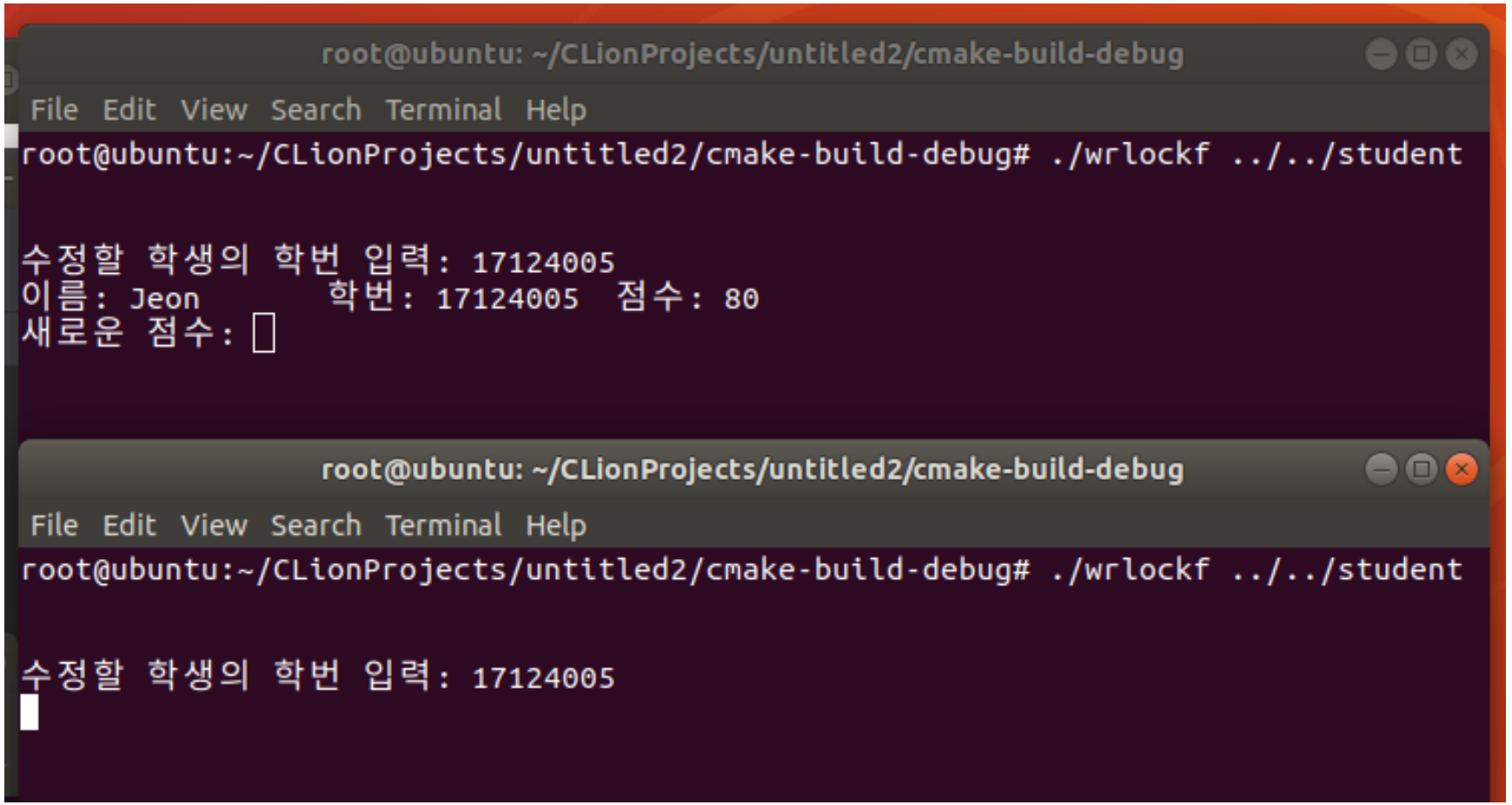
❖ 실행 명령어

- gcc –o wrlockf.c
- ./wrlockf student
- ./wrlockf student

실습 문제 2 – 레코드 영역 동시 수정

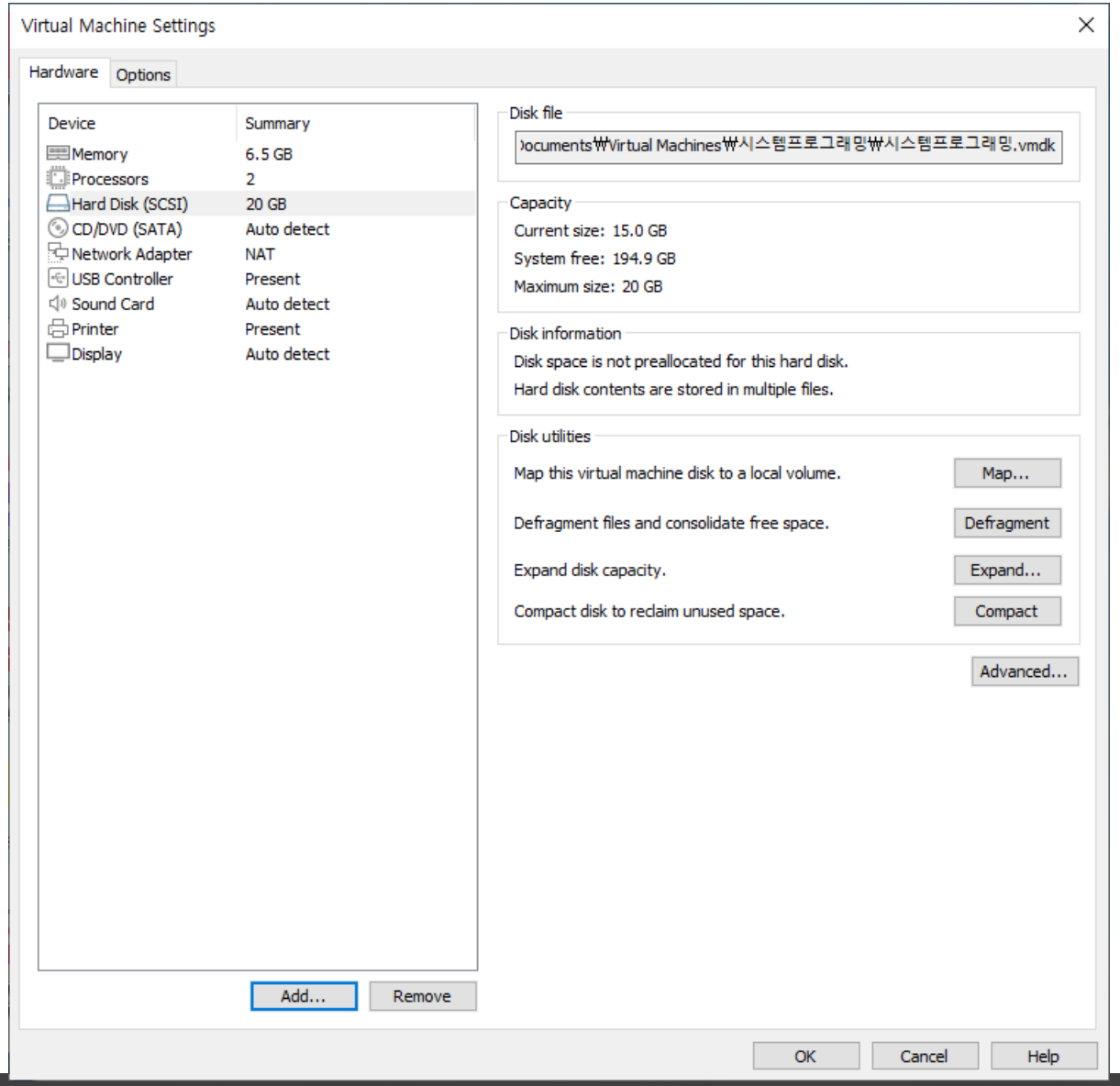
- 실행 결과

Terminal GCC



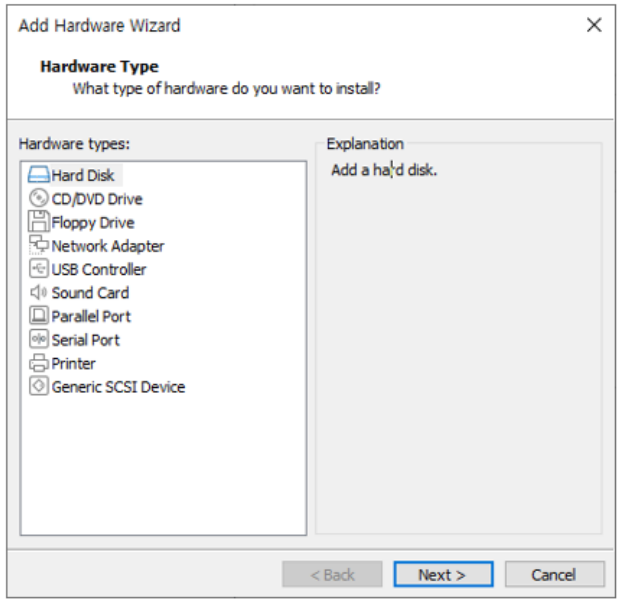
실습 문제 3 – 권고 잠금과 강제 잠금

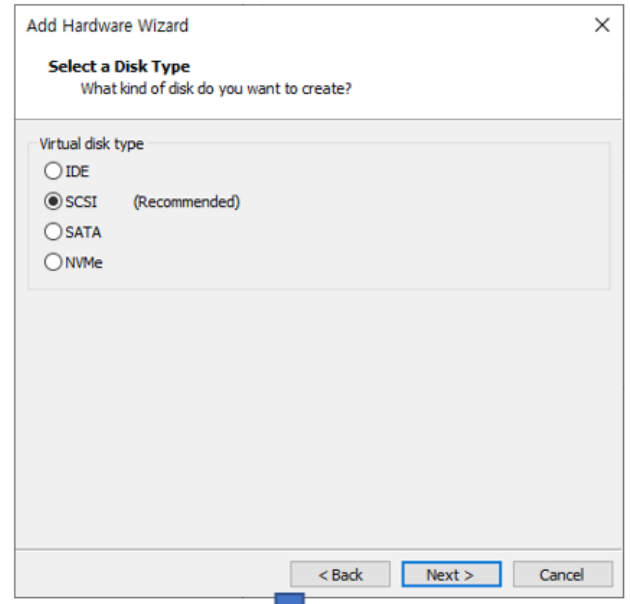
- 실습 진행을 위한 추가 설정
  - ✓1. 가상머신 하드디스크 추가

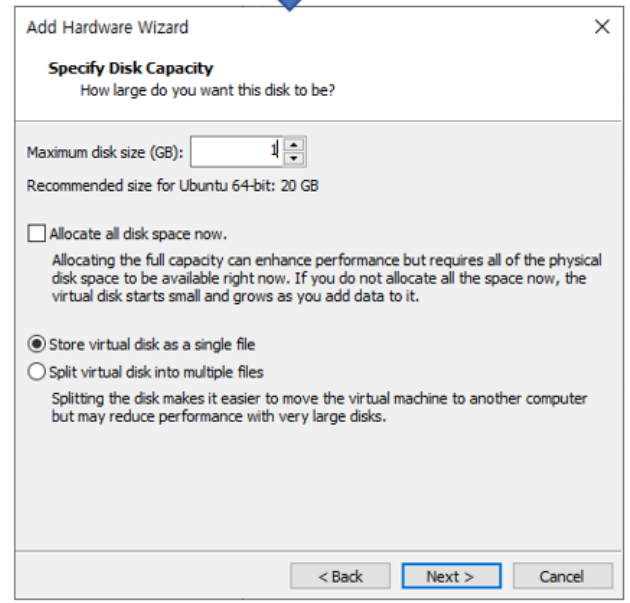


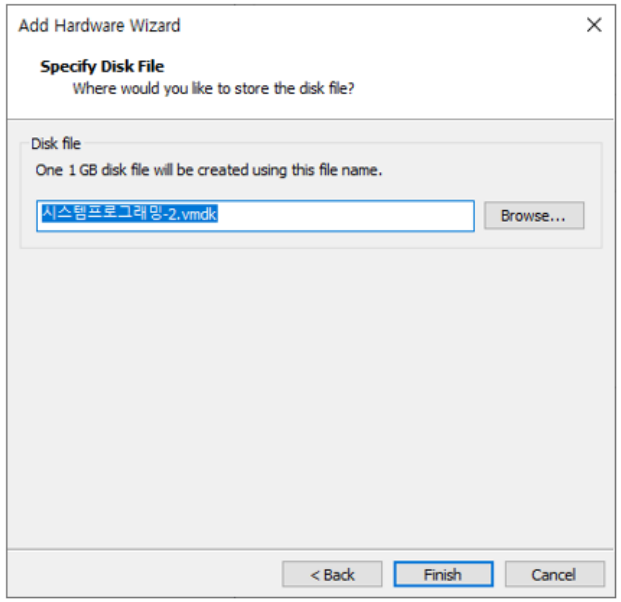
실습 문제 3 – 권고 잠금과 강제 잠금

- 실습 진행을 위한 추가 설정
  - ✓하드디스크 클릭 후 Add.. 클릭









실습 문제 3 – 권고 잠금과 강제 잠금

- 생성한 하드디스크 파티션 생성
  - ✓ sudo fdisk /dev/sdb 후
  - ✓ n 입력 --> 엔터 --> 엔터 --> 엔터 --> q 입력

```
jaehyuk@ubuntu:~$ sudo fdisk /dev/sdb
[sudo] password for jaehyuk:

Welcome to fdisk (util-linux 2.31.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0x1352c436.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1):
First sector (2048-2097151, default 2048):
Last sector, +sectors or +size{K,M,G,T,P} (2048-2097151, default 2097151):

Created a new partition 1 of type 'Linux' and of size 1023 MiB.

Command (m for help): q

jaehyuk@ubuntu:~$
```

실습 문제 3 – 권고 잠금과 강제 잠금

- 파일 시스템 생성

```
sudo mke2fs /dev/sdb
mke2fs 1.44.1 (24-Mar-2018)
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: 99289f33-6789-4a12-ab30-8361fdc77ff3
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done

jaehyuk@ubuntu:~$
```

- 마운트 폴더 생성

```
jaehyuk@ubuntu:~$ cd /mnt/
jaehyuk@ubuntu:/mnt$ sudo mkdir hdd
```

실습 문제 3 – 권고 잠금과 강제 잠금

- 마운트

```
jaehyuk@ubuntu:~$ sudo mount -o mand /dev/sdb /mnt/hdd/
jaehyuk@ubuntu:~$
```

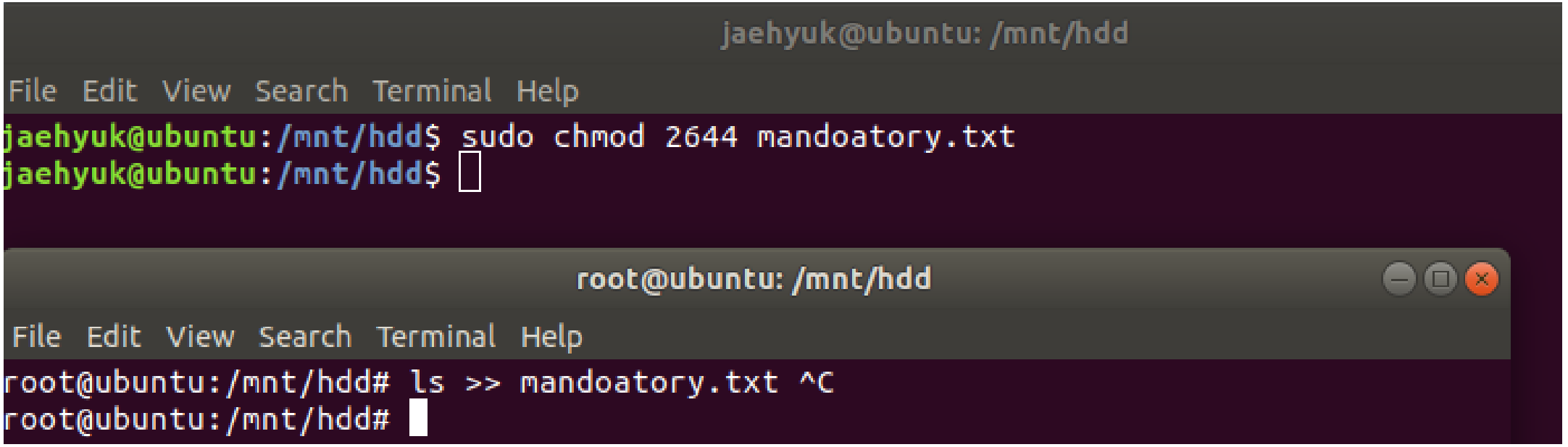
- 마운트 후, 기존 실습 파일 복사

```
jaehyuk@ubuntu:~/ch6$ sudo cp * /mnt/hdd
[sudo] password for jaehyuk:
jaehyuk@ubuntu:~/ch6$
```

```
jaehyuk@ubuntu:/mnt/hdd$ ls
5    7    8    advisory.txt  file_lock.c  mandoatory.txt  rdlock.c  stdb1    wrlock    wrlockf
5.c  7.c  8.c  file_lock     lost+found   rdlock          record.h  student  wrlock.c  wrlockf.c
jaehyuk@ubuntu:/mnt/hdd$
```

실습 문제 3 – 권고 잠금과 강제 잠금

- 실습 진행
  - ✓ 터미널 2개 연 후, 1번 터미널에서 mandoatory.txt 권한 수정(sudo chmod 2644 mandoatory.txt)
  - ✓ 2번 터미널은 관리자 권한 실행(sudo su)



실습 문제 3 – 권고 잠금과 강제 잠금

- 권고 잠금과 강제 잠금의 차이에 대하여 확인하시오.

✓참조 자료 : 강의자료 24번 슬라이드

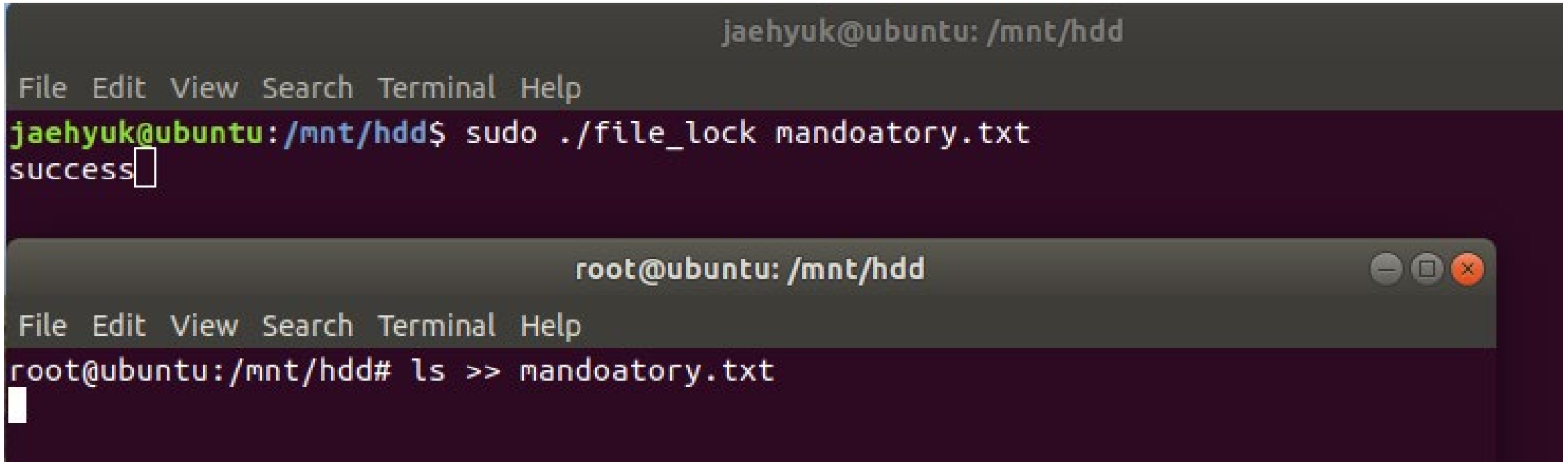
1. file_lock.c 파일을 작성하고 실행하시오.
2. 터미널을 사용하여 mandatory.txt, advisory.txt 파일을 2644, 0644 권한으로 생성하시오.
3. file_lock을 실행하여 각 txt파일의 argument 로 넣었을때 실행 결과를 확인하시오.
4. 예외처리
   - lockf() 함수를 사용하여 파일 잠금 처리를 수행. 만약, 잠금을 할 수 없다면 -1을 반환

❖ 실행 명령어

- gcc –o file_lock.c
- ./file_lock mandatory.txt
- ls >> mandatory.txt
- ./file_lock advisory.txt
- ls >> advisory.txt

실습 문제 3 – 파일 열기

- 실행 결과



과제 1 – 원하는 잠금을 설정할 수 있는 함수 locks()를 작성

1. 원하는 잠금을 설정할 수 있는 함수 locks()를 작성하시오. 이 함수는 다음과 같이 매개변수로 받은 파일의 특정 영역 (레코드) 에 대해 원하는 잠금을 설정한다

```
#include <sys/types.h>
#include <fcntl.h>
int locks(int fd, int type, off_t offset, int whence, off_t len);
```

- 이 함수의 매개변수는 다음과 같다.
  - ✓ type: 설정할 잠금의 종류로 F_RDLCK 혹은 F_WRLCK
  - ✓ offset: whence를 기준으로 바이트 오프셋
  - ✓ whence: 기준점으로 SEEK_SET, SEEK_CUR, SEEK_END
  - ✓ len: 바이트수 (0이면 파일 끝까지
- Hint: 실습 3번 file_lock.c (fcntl) 을 참조하여 현재 잠금 상태를 확인 할 수 있습니다.



과제 2 – 원하는 잠금을 검사할 수 있는 함수 lockt()를 작성

2. 이 함수는 다음과 같이 매개변수로 받은 파일의 특정 영역에 원하는 잠금을 할 수 있는지 검사하여 가능하면 0을, 불가능하면 불가능하게 만드는 기존 잠금을 소유하는 프로세스의 번호를 반환한다

```
#include <sys/types.h>
#include <fcntl.h>
pid_t lockt(int fd, int type, off_t offset, int whence, off_t len)
```

- 함수의 매개변수의 설명은 과제 1과 같다.
- Hint: 실습 3번 file_lock.c (getpid()) 를 사용하여 프로세스 번호를 확인할 수 있습니다.



E N D





School of Computer Software at Daegu Catholic University © 2021